

DADER

Intro

ER classica

La **Entity Resolution (ER)** è un **compito centrale nel data integration**.

Date due tabelle relazionali A e B , ciascuna **tupla** $(a \in A, b \in B)$ un'entità descritta come **insieme di coppie attributo-valore** $\{(att_i, val_i)\}$.

L'obiettivo dell'ER è **identificare le coppie** $(a, b) \in A \times B$ **che si riferiscono allo stesso oggetto reale**.

Pipeline Classica

Fasi:

1. **Blocking** → riduce il numero di confronti eliminando coppie chiaramente non in match;
2. **Matching** → classifica le coppie rimaste in match/non-match tramite un modello.

Training Data

Un **training set** $(\mathcal{D}, \mathcal{Y})$ contiene:

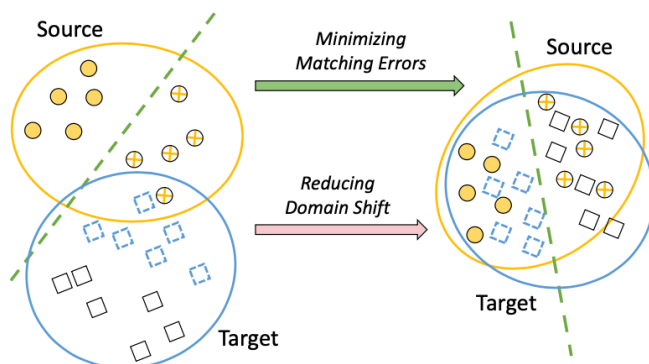
- coppie di entità $(a, b) \in \mathcal{D} \subset A \times B \rightarrow$ ciascuna coppia è associata ad un'etichetta che ne specifica la relazione
- etichette binarie $y \in \mathcal{Y} \rightarrow y = 1$ match, $y = 0$ non-match

Domain Adaptation

Domanda centrale del paper, coincide con il **concetto di domain adaptation (DA)**:

Possiamo usare un dataset ER sorgente ben etichettato per allenare un modello che funzioni su un dominio target con zero o poche etichette?

Il **problema principale** è il **domain shift** – **differenze di distribuzione fra source e target** – che **degrada le prestazioni** del modello.



L'obiettivo della **DA** è **apprendere rappresentazioni domain-invariant e discriminanti**, in grado di ridurre lo shift e migliorare l'accuratezza sul target.

Framework DADER

DADER (Domain Adaptation for Deep Entity Resolution) **unifica** diversi **approcci di DA** ed è composto da **3 moduli**:

1. **Feature Extractor** ($\mathcal{F}$) → **converte** le **coppie** (a, b) in vettori altamente dimensionali, cioè in **features**, x
2. **Matcher** ($\mathcal{M}$) → **classificatore binario** che, date in input le features, predice se sono in match/non-match
3. **Feature Aligner** ($\mathcal{A}$) → **componente chiave** della DA, **allevia** gli effetti del **domain shift**:
 - aggiusta $\mathcal{F}$ per allineare le distribuzioni di source e target (domain-invariance ++)
 - aggiorna $\mathcal{M}$ per minimizzare gli errori di match (discriminanza ++)

Domande di design analizzate dal paper

- Quali scelte di progettazione per DA aiutano davvero l'ER?

Scelte di design per ciascun componente:

- $\mathcal{F}$: reti neurali e modelli linguistici pre-addestrati
- $\mathcal{M}$: reti neurali deep
- $\mathcal{A}$: discrepancy/adversarial/reconstruction-based methods

- DA è davvero utile in ER?

Impostazione degli scenari:

- unsupervised (senza etichette nel target)
- semi-supervised (alcune etichette target)

Modules	Categorization	
Feature Extractor ($\mathcal{F}$)	(I) Recurrent neural network (RNN)	
	(II) Pre-trained language models (LMs)	
Matcher ($\mathcal{M}$)	Multi-layer Perceptron (MLP)	
Feature Aligner ($\mathcal{A}$)	(1) Discrepancy-based	(a) MMD (b) K -order
	(2) Adversarial-based	(c) GRL (d) InvGAN (e) InvGAN+KD
	(3) Reconstruction-based	(f) ED

Deep ER

Una pipeline di Deep ER utilizza due componenti:

- $\mathcal{F} \rightarrow$ genera feature d -dimensionali

$$\mathbf{x} = \mathcal{F}(a, b) : A \times B \rightarrow \mathbb{R}^d$$

- $\mathcal{M} \rightarrow$ predice la probabilità di match

$$\hat{y} = \mathcal{M}(\mathbf{x}) = \mathcal{M}(\mathcal{F}(a, b))$$

Durante l'addestramento, $\mathcal{F}$ e $\mathcal{M}$ sono ottimizzati tramite gradient descent.

DA per Deep ER

Dato:

- un dataset sorgente etichettato $(\mathcal{D}^S, \mathcal{Y}^S) = \{(a^S, b^S, y^S)\}$
- un dataset target non etichettato $\mathcal{D}^T = \{(a^T, b^T)\}$

L'obiettivo è trovare $\mathcal{F}$ ed $\mathcal{M}$ che producano risultati accurati sul target. Il domain shift influisce sia sulle feature estratte sia sulla qualità del matcher.

Ruolo del Feature Aligner

L'idea è di imparare direttamente dalle istanze quale sia il miglior modo di generare ed allineare le features. Per fare ciò, $\mathcal{A}$:

- guida $\mathcal{F}$ nella generazione di features adatte
- aggiorna $\mathcal{M}$ di conseguenza

Proprietà di domain-invariant

Si vuole che le **source features** $\mathbf{x}^S$ e le **target features** $\mathbf{x}^T$ siano il **più vicine possibili**.
Affinché ciò avvenga

$$\mathcal{A}(\mathbf{x}^S, \mathbf{x}^T) : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^d$$

è utilizzato per produrre una **funzione di alignment loss** $\mathcal{L}_A$.

Proprietà di discriminanza

Si utilizza una **funzione di matching loss** che **misura la differenza tra i valori predetti e quelli di origine**:

$$\mathcal{L}_M = \text{loss}(\{(\hat{y}, y^S)\})$$

dove loss è una funzione tipo cross entropy.

Obiettivo di ottimizzazione complessivo

Trovare un $\mathcal{F}^*$ ed un $\mathcal{M}^*$ che minimizzino $\mathcal{L}_A$ ed $\mathcal{L}_M$:

$$\mathcal{F}^*, \mathcal{M}^* = \arg \min_{\mathcal{F}, \mathcal{M}} \text{aggregate}(\mathcal{L}_A, \mathcal{L}_M)$$

dove aggregate è una funzione determinata da diverse strategie di feature alignment.

Per raggiungere lo scopo si **sfrutta la back-propagation per aggiornare i parametri** di $\mathcal{F}$ ed $\mathcal{M}$.

Design del Feature Aligner

La **differenza tra i metodi** sta nel modo in cui definiscono $\mathcal{L}_A$.

Metodi discrepancy-based

Questa famiglia di metodi misura $\mathcal{L}_A$ calcolando la **discrepanza di distribuzione** tra source e target. Sono semplici e stabili, in quanto ottimizzano solo $\mathcal{F}$ ed $\mathcal{M}$.

Durante il training:

- $\mathcal{F}$ genera le feature ed $\{\mathbf{x}^T\}$, con rispettivi spazi p^S e p^T , a partire dai domini $\mathcal{D}^S$ e $\mathcal{D}^T$
- $\mathcal{A}$ calcola la misura di discrepanza $\mathcal{L}_A$ tra p_S e p_T
- $\mathcal{M}$ calcola $\mathcal{L}_M$ sul dominio etichettato $\mathcal{D}^S$

$$\mathcal{L}_M = E_{(x^S, y^S) \sim (\mathcal{D}^S, \mathcal{Y}^S)} [\mathcal{L}_{CE}(\mathcal{M}(\mathcal{F}(x^S)), y^S)]$$

dove $\mathcal{L}_{CE}$ è la cross-entropy loss ed $E[\cdot]$ è l'expectation

La funzione obiettivo minimizza $\mathcal{L}_A$ ed $\mathcal{L}_M$:

$$\min_{\mathcal{F}, \mathcal{M}, \mathcal{A}} V(\mathcal{F}, \mathcal{M}, \mathcal{A}) = \mathcal{L}_M(\mathcal{F}, \mathcal{M}) + \beta \mathcal{L}_A(\mathcal{F}, \mathcal{A})$$

dove β è un **iperparametro** che controlla il **trade-off tra matching e confusione di dominio**. Il **valore** viene **calcolato considerando l'F1 score maggiore** sul validation set $\mathcal{D}_{\text{val}}^T$ di diversi dataset.

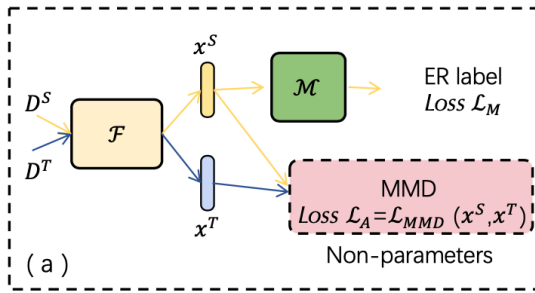
Maximum Mean Discrepancy (MMD)

Misura la distanza tra le medie delle due distribuzioni in uno spazio kernel Hilbert (RKHS). Quando la distribuzione tra p_S e p_T è la stessa, $\mathcal{L}_{MMD}$ è nulla.

Funzione di loss:

$$\mathcal{L}_{MMD} = \sup_{\|\phi\|_H \leq 1} \|E_{\mathbf{x}^S \sim p_S}[\phi(\mathbf{x}^S)] - E_{\mathbf{x}^T \sim p_T}[\phi(\mathbf{x}^T)]\|_H^2$$

dove ϕ è la **funzione kernel** che effettua il **mapping** ed $\|\phi\|_H \leq 1$ definisce un insieme di funzioni in RKHS(H).

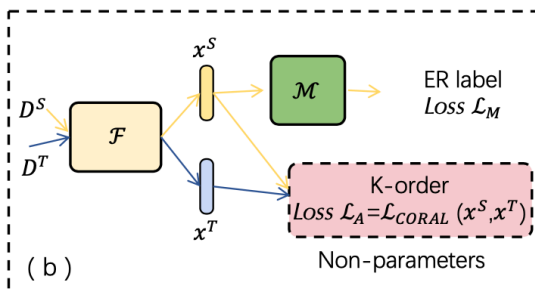


K-orders / CORAL

Considera le **covarianze dei domini** e calcola la distanza tramite la funzione di loss:

$$\mathcal{L}_{CORAL} = \frac{1}{4d^2} \|C_S - C_T\|_F^2$$

dove d è la dimensione di $\mathbf{x}$, $C_S(C_T)$ è la matrice di covarianza tra le feature $\mathbf{x}^S(\mathbf{x}^T)$ ed $\|\cdot\|_F^2$ indica la matrice quadratica della norma di Frobenius.



Metodi adversarial-based

Questa famiglia di metodi misura $\mathcal{L}_A$ utilizzando un **classificatore di dominio binario** – implementato tramite strati full-connected – e un **processo di adversarial training** stile GAN: $\mathcal{F}$ è addestrato a **generare features fuorvianti** per il classificatore.

La funzione obiettivo è:

$$\min_{\mathcal{F}, \mathcal{M}} \max_{\mathcal{A}} V(\mathcal{F}, \mathcal{M}, \mathcal{A}) = \mathcal{L}_M(\mathcal{F}, \mathcal{M}) + \beta \mathcal{L}_A(\mathcal{F}, \mathcal{A})$$

dove β è un **iperparametro** che controlla il **trade-off tra matching e classificazione** del dominio ed

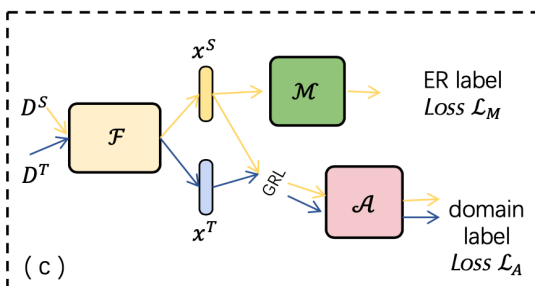
$$\mathcal{L}_A = E_{x^S \sim \mathcal{D}^S} \log \mathcal{A}(\mathcal{F}(x^S)) + E_{x^T \sim \mathcal{D}^T} \log (1 - \mathcal{A}(\mathcal{F}(x^T)))$$

Gradient Reversal Layer (GRL)

Aggiunge uno strato gradient reversal tra $\mathcal{F}$ ed $\mathcal{A}$ che realizza l'addestramento avverso. Durante la back-propagation, il gradiente di $\mathcal{A}$ moltiplica una certa costante negativa del gradiente di questo strato addizionale.

La funzione obiettivo è:

$$\max_{\mathcal{F}} \min_{\mathcal{A}} \mathcal{L}_A = -E_{x^S \sim \mathcal{D}^S} \log (\mathcal{A}(\mathcal{F}(x^S))) - E_{x^T \sim \mathcal{D}^T} \log (1 - \mathcal{A}(\mathcal{F}(x^T)))$$



Inverted labels GAN (InvGAN)

Adatta la dinamica GAN: gli x^S sono i **dati reali**, i x^T sono i **dati** generati da $\mathcal{F}$ il più **simili** possibile ad x^S ed $\mathcal{A}$ è il **discriminatore**.

Il processo di training si articola in:

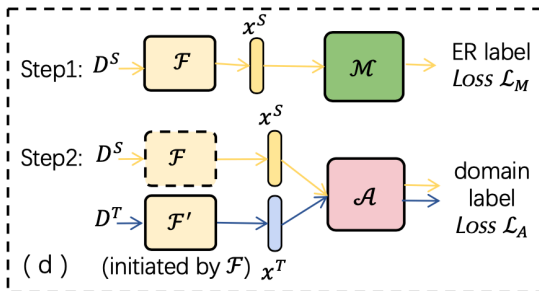
- un primo addestramento di $\mathcal{F}$ ed $\mathcal{M}$ utilizzando una source di dati etichettati
- si clona un $\mathcal{F}'$ a partire da $\mathcal{F}$, e si addestra per produrre gli esempi falsi x^T attraverso la funzione:

$$\min_{\mathcal{F}'} \mathcal{L}_{\mathcal{F}'} = -E_{x^T \sim \mathcal{D}^T} \log(\mathcal{A}(\mathcal{F}'(x^T)))$$

- $\mathcal{A}$ viene addestrata per distinguere le feature reali x^S (prodotte da $\mathcal{F}$) da quelle false x^T , attraverso la funzione:

$$\min_{\mathcal{A}} \mathcal{L}_{\mathcal{A}} = -E_{x^S \sim \mathcal{D}^S} \log(\mathcal{A}(\mathcal{F}(x^S))) - E_{x^T \sim \mathcal{D}^T} \log(1 - \mathcal{A}(\mathcal{F}'(x^T)))$$

Un **problema** che si è osservato è che **le feature prodotte da $\mathcal{F}'$ potrebbero perdere la proprietà di discriminanza**, portando ad un peggioramento delle performance di $\mathcal{M}$.



InvGAN+KD

Questo metodo nasce dalla volontà di alleviare la problematica di InvGAN: **aggiunge KD per impedire perdita di discriminanza**.

$$\mathcal{L}_{KD} = t^2 \times E_{x^S \sim \mathcal{D}^S} \sum -\text{softmax}(\mathcal{M}(\mathcal{F}(x^S))/t) \times \log(\text{softmax}(\mathcal{M}(\mathcal{F}'(x^S))/t))$$

dove t è **iperparametro** che **determina quanto** il modello **attenua le distribuzioni**.

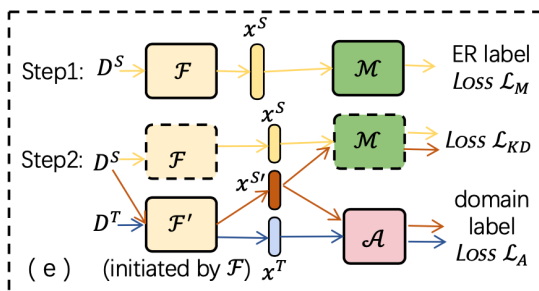
Analogamente a prima, vengono utilizzate le feature generate da $\mathcal{F}'$ ed $\mathcal{F}$ per calcolare le funzioni obiettivo per l'addestramento di $\mathcal{A}$:

$$\min_{\mathcal{A}} \mathcal{L}_{\mathcal{A}} = -E_{x^S \sim \mathcal{D}^S} \log(\mathcal{A}(\mathcal{F}'(x^S))) - E_{x^T \sim \mathcal{D}^T} \log(1 - \mathcal{A}(\mathcal{F}'(x^T)))$$

ed $\mathcal{F}'$:

$$\min_{\mathcal{F}'} \mathcal{L}_{\mathcal{F}'} = -E_{x^T \sim \mathcal{D}^T} \log(\mathcal{A}(\mathcal{F}'(x^T))) + \mathcal{L}_{KD}$$

L'**obiettivo** è quello di ottenere delle **performance** di $\mathcal{F}'$ ed $\mathcal{M}$ **analoghe** a quelle di $\mathcal{F}$ ed $\mathcal{M}$.



Metodi reconstruction-based

Questa famiglia di metodi misura $\mathcal{L}_{\mathcal{A}}$ introducendo un **task di ricostruzione** ausiliario che consente di **estrarre uno spazio nascosto condiviso fra i due domini**.

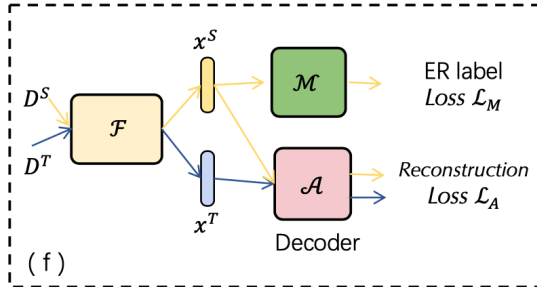
Encoder-Decoder (ED)

La pipeline del processo di training consiste in:

- x^S ed x^T sono processate da $\mathcal{F}$ per generare le features $\mathcal{F}(x^S)$ ed $\mathcal{F}(x^T)$
- le feature $\mathcal{F}(x^S)$ sono fornite in input ad $\mathcal{M}$, cos' da poter calcolare la funzione di loss $\mathcal{L}_M$
- decodifica le feature provando a ricostruire l'input, necessario per il calcolo della funzione:

$$\mathcal{L}_{REC} = E_{x \sim \mathcal{D}^S \cup \mathcal{D}^T} [\mathcal{L}_{CE}(\mathcal{A}(\mathcal{F}(x)), x)]$$

utilizzata poi per l'aggiornamento dei parametri di $\mathcal{F}$ ed $\mathcal{M}$.



Design del Feature Extractor

Le scelte di design per $\mathcal{F}$ rientrano in due categorie principali

Recurrent Neural Network (RNN)

RNN è una tecnica utilizzata per **codificare del testo con lunghezza variabile in features**.

Per ciascun k -esimo attributo della coppia (a, b) viene effettuato un embedding, ottenendo k **attribute embeddings** $\{e_1^a, \dots, e_k^a\}$ per ciascuna entità.

Successivamente, viene **calcolata una misura di similarità** tra i due insiemi di attribute embeddings attraverso due **modalità**:

1. viene calcolato un vettore di similarità s_i per ogni coppia (e_i^a, e_i^b) , poi vengono uniti per ottenere un unico s
2. vengono aggregati gli attribute embedding in un unico embedding, la coppia (e^a, e^b) viene quindi utilizzata per calcolare il singolo embedding di similarità s

Pre-trained Language Models (LMs)

Questa classe include tecnologie quali **Bert**, RoBerta, DistilBert.

Design del Matcher

Il **classificatore fully-connected** riceve in input una feature x e ritorna un intero 1/0 che indica match/non-match.

Valutazione

Datasets

- DeepMatcher
- Magellan
Coprono una varietà di domini
- 4 dataset di WDC con dati provenienti da siti web di e-commerce

Setup

Il dataset target $\mathcal{D}^T$ è stato suddiviso, con ratio 1:9, in:

- **validation set** $\mathcal{D}_{val}^T \rightarrow$ utilizzato anche per il **calcolo degli iperparametri**
- **test set** $\mathcal{D}_{test}^T \rightarrow$ **omesso nel processo training** nella DA

Per ciascun metodo di DA per ER:

1. DADER viene addestrato, dividendo il processo in 40 epoche
2. per ciascuna epoca sono calcolate le performance sul $\mathcal{D}_{val}^T$
3. viene selezionato il modello, con le miglior performance generali, da cui prendere $\mathcal{F}$ ed $\mathcal{M}$
4. vengono utilizzati $\mathcal{F}$ ed $\mathcal{M}$ sul $\mathcal{D}_{test}^T$

Metriche

La valutazione utilizza l'**F1 score**: armonica della **media tra precision e recall** calcolate sulle coppie classificate come match.

In particolare:

- **TP (True Positive)** → coppie realmente in match che $\mathcal{M}$ classifica come tali
- **FP (False Positive)** → coppie non in corrispondenza che $\mathcal{M}$ classifica come in match
- **FN (False Negative)** → coppie effettivamente in match che $\mathcal{M}$ non identifica

Le metriche di **precision e recall** sono calcolate come:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

Da esse deriva l'**F1-score**:

$$F1 = 2 \cdot P \cdot \frac{R}{P + R}$$

Approcci alternativi

Oltre ai metodi di DA analizzati, vengono utilizzati due **baseline** significative:

- **NoDA** → il modello viene addestrato sul dominio source senza adattamento al target
- **Reweight** → non modifica lo spazio delle feature, ma ripesa le coppie di entità del source in base alla loro similarità con il target. In particolare:
 1. si convertono le coppie testuali in vettori, tramite embedding pre-addestrati
 2. si calcola la similarità tra feature source e target
 3. ogni coppia source riceve un peso proporzionale a tale similarità
 4. $\mathcal{M}$ è addestrato tenendo conto dei pesi

Reweight è **sensibile alla** qualità della **misura di similarità**, che diventa difficile da stimare in scenari con forte domain shift.

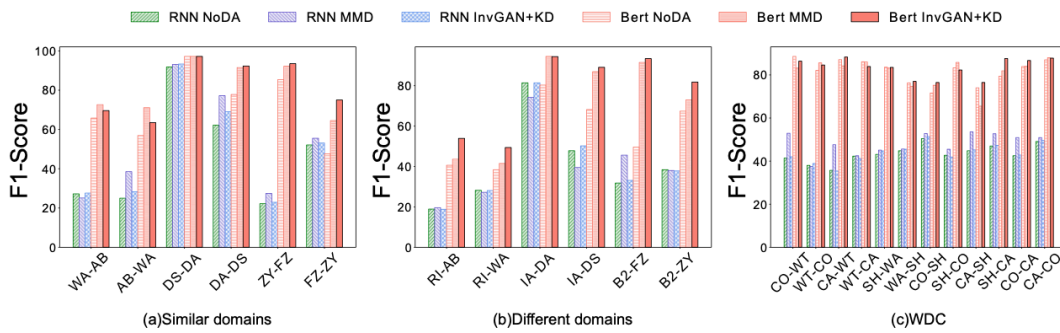
Risultati

I risultati sono analizzati in base alla **relazione tra i domini source e target**:

1. **Domini simili** → DA porta miglioramenti ovvi e modesti, poiché il domain shift da correggere è modesto
2. **Domini differenti** → guadagni della DA meno immediati ma più significativi, perché compensa differenze sostanziali nelle distribuzioni dei dati

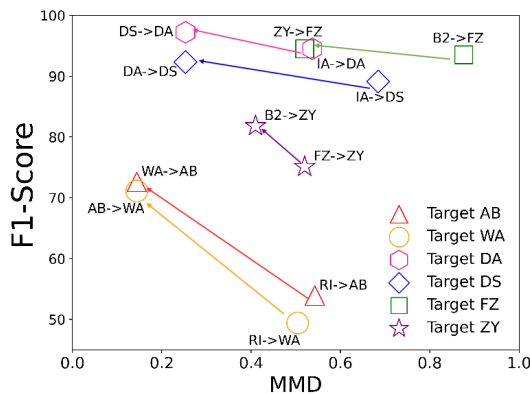
Da queste conclusioni emerge che la **prima osservazione** importante:

la DA è utile in entrambi gli scenari, anche se l'entità del miglioramento varia.



L'analisi è stata approfondita misurando la distanza tra domini tramite MMD, giungendo alla **seconda osservazione** importante:

quando source e target hanno **bassa MMD**, la DA raggiunge un **F1-score più elevato**. Ciò suggerisce una **strategia pratica per la scelta del dataset source più adatto a un target specifico**.



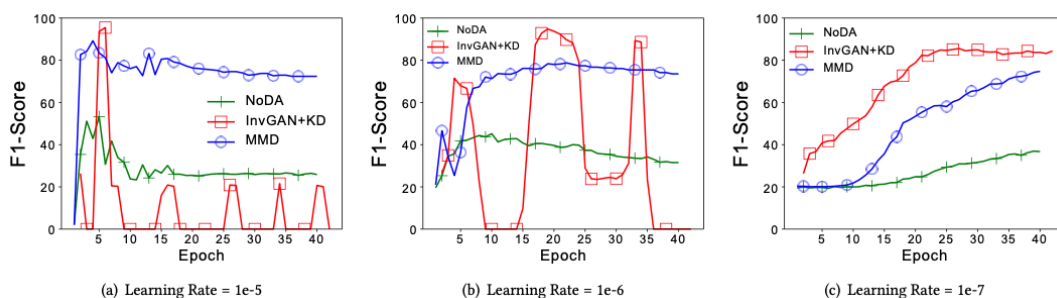
Valutazione del Feature Aligner

L'analisi distingue due categorie di scenari:

1. **Casi di successo** – DA migliore rispetto a NoDA → il **confronto** tra MMD e InvGAN+KD mostra che:
 - **MMD converge stabilmente** con un numero sufficiente di epoche;
 - **InvGAN+KD** ha un andamento più **oscillante**, è più **sensibile agli iperparametri** e, per migliorare, richiede un $\mathcal{D}_{val}^T$ di dimensioni più ridotte

Da ciò deriva la **terza osservazione** importante:

i **metodi discrepancy-based convergono** in modo regolare, mentre i **metodi adversarial-based** tendono a oscillare e possono diventare **instabili**. **Ridurre il learning rate può mitigare l'oscillazione**, ma al costo di **tempi di addestramento più lunghi**.



2. **Casi di fallimento** – DA poco o per nulla migliore rispetto a NoDA → il **confronto** tra InvGAN, ED e GRL mostra che:
 - l'**F1-score di InvGAN** **cala rapidamente** o resta **instabile** perché il modello privilegia l'allineamento delle distribuzioni **trascurando la discriminanza**, danneggiando le capacità di $\mathcal{M}$
 - **ED** ha **performance scarse** quasi in tutti i casi, poiché **fatica a ricostruire** il testo originale
 - **GRL** è **generalmente efficace**, ma **può fallire quando il training è instabile**, inoltre, le **performance dipendono fortemente** dalla scelta degli **iperparametri**.

Queste analisi portano alla **quarta osservazione** importante:

non tutti i metodi di DA sono adatti ai problemi di ER. È essenziale **garantire features domain-invariance e discriminanti**, il corretto **bilanciamento** fra le due è **cruciale**.

Valutazione del Feature Extractor

La **quinta osservazione** importante evidenzia che:

$\mathcal{F}$ basati su **RNN** offrono **guadagni limitati**, l'efficacia della DA **dipende dalla trasferibilità** delle rappresentazioni, e gli **LM pre-addestrati** risultano nettamente **superiori**.

Comparazione

Con approcci già esistenti

Reweight si concentra su un **approccio a livello di istanza** che risulta essere poco efficace quando la similarità tra source e target è difficile da misurare oppure quando la quantità dei dati sorgente è ridotta. **DADER opera a livello di feature** e non ha questi limiti.

Da qui la **sesta osservazione** importante:

i **metodi DA basati sulle feature superano quelli basati sulle istanze**, poiché permettono di apprendere rappresentazioni realmente condivise tra i domini.

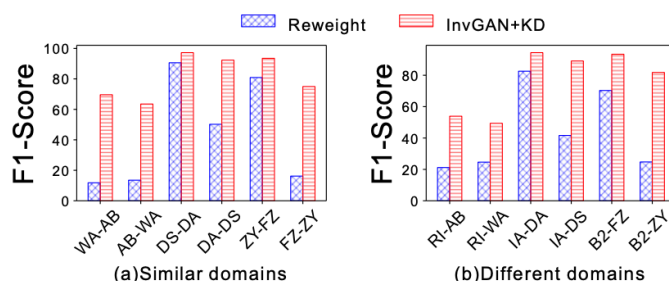


Figure 10: Comparison between DADER and Reweight.

Usando etichette target

La **settima osservazione** importante evidenzia che:

in scenari semi-supervisionati, anche **un piccolo numero di etichette target** è **sufficiente per mantenere (o migliorare) i risultati** ottenuti dalla DA. In molti casi, il modello supera metodi all'avanguardia come DeepMatcher e Ditto.